Segurança de Dados

Assignment 4: Block ciphers and modes of operation

Hugo Soares

30 de outubro de 2026

Questão 1

Ideia. Em CTR o keystream é $E_K(\text{nonce}\|\text{contador})$ e a cifra é só $C = P \oplus K$. O AES é usado para gerar o keystream, mas a operação final continua a ser XOR. Logo, para trocar P por P' sem saber K, anula-se P e põe-se P': $C' = C \oplus P \oplus P'$.

Código.

```
p  = b"PLC=07;CMD=SET;NAOH_PPM=00100;"
pp = b"PLC=07;CMD=SET;NAOH_PPM=11100;"
forged = xor(q1_ct, xor(p, pp))
```

Resultado. Confirmei com a chave que o ciphertext dado decifra para o valor esperado (`00100`), e a forja é:

```
Q1 forged: 51073653f3851c51af06c462d293862e93941666f8d561dbeeddc89cd972
Q1 changed bytes: [(24, 0xef, 0xee), (25, 0xdc, 0xdd)]
Q1 decrypts to: PLC=07;CMD=SET;NAOH_PPM=11100;
```

Só mudam os bytes 24 e 25, que correspondem aos dois primeiros dígitos de `00100` → `11100`: `0x30`⊕`0x31` = `0x01`, por isso `ef`→`ee` e `dc`→`dd`. O contador do segundo bloco não interfere porque a alteração é feita byte a byte dentro do mesmo bloco.

Q1.b. Não. A migração de ChaCha20 para AES-128-CTR não corrige a malleability: ambos são cifras de fluxo. O AES só é usado para produzir o keystream; a operação que combina com o plaintext continua a ser XOR e não há MAC. Como o formato é público, o atacante sabe exatamente que bytes alterar. O que resolve o problema é um modo autenticado (AES-GCM, ChaCha20-Poly1305) ou encrypt-then-MAC; a escolha do algoritmo de bloco é irrelevante para esta falha. (O nonce único continua a ser necessário, mas isso é a proteção contra reutilização de keystream, não contra malleability.)

Questão 2

Ideia. ECB cifra cada bloco de 16 bytes de forma independente e determinística. Então dois blocos de plaintext iguais dão sempre o mesmo bloco de ciphertext. Sem decifrar nada, basta comparar os ciphertexts bloco a bloco.

Código.

```
blocks = [[c[k:k+16] for k in range(0, len(c), 16)] for c in q2_cts]
for i in range(len(q2_cts)):
    for j in range(i + 1, len(q2_cts)):
        same = q2_cts[i] == q2_cts[j]
        eq = sum(a == b for a, b in zip(blocks[i], blocks[j]))
        print(f"C{i+1} vs C{j+1}: identical={same}, equal blocks={eq}")
```

Resultado. Os quatro ciphertexts têm 64 bytes (4 blocos):

Par	Iguais?	Blocos iguais
C_1 vs C_3	sim	4/4
todos os outros pares	não	0/4

Ou seja, C_1 e C_3 são exatamente o mesmo comando cifrado; C_2 e C_4 são distintos de todos os outros. Não é preciso a chave: a igualdade dos ciphertexts prova a igualdade dos plaintexts, porque ECB é determinístico.

O que ECB revela. Revela igualdade de blocos, prefixos comuns e repetições na estrutura da mensagem. Como os comandos têm formato fixo (`PLC=<id>;CMD=SET;NAOH_PPM=<valor>;`), dois comandos com o mesmo prefixo de 16 bytes produzem o mesmo primeiro bloco. Confirmei isto com uma chave de demonstração e dois valores diferentes:

```
Q2 demo C1: 3bee1322ae4c14c822683a2c86d9a400
            d5873fd250bf85cb298bebe675469a3e
Q2 demo C2: 3bee1322ae4c14c822683a2c86d9a400
            7e5a853b32bf93b30602b59db4ec980b
Q2 demo same first block: True
```

Além disso, ECB permite cortar e colar blocos, reordenar blocos e montar um dicionário de blocos frequentes. Para tráfego de PLC com mensagens muito estruturadas isto é inaceitável: um atacante distingue comandos iguais, deteta padrões e pode manipular blocos sem conhecer a chave.

Questão 3

Ideia. Em CBC, $c_0 = E_K(P_0 \oplus iv_0)$ e, para uma mensagem injetada, $c_1 = E_K(P \oplus iv)$. Como o IV é um contador previsível, sabemos o IV de cada query antes de a enviar. Se fizermos $P \oplus iv = P_0 \oplus iv_0$, então $c_1 = c_0$ e o bloco cifrado é o mesmo. Resolvendo em ordem a P: $P = P_0 \oplus iv_0 \oplus iv$.

O bloco secreto tem 16 bytes e é `PLC=<id>;CMD=SET;N`; só os dois dígitos do `<id>` são desconhecidos. São 100 hipóteses, por isso testo cada uma com o oráculo e comparo o primeiro bloco com c_0. Cada query gasta um IV novo, por isso o teste tem de usar o IV dessa query, não sempre iv_1.

Código. O oráculo é só para emular o centro de controlo:

```
def test_block(guess, iv):
    block = b"PLC=" + guess + b";CMD=SET;N"
    query = xor(block, xor(iv0, iv))   # P xor iv == block xor iv0
    return cbc_encrypt(key, iv, query)[:16] == c0

counter = int.from_bytes(iv1, "big")
matches = []
for i in range(100):
    iv = counter.to_bytes(16, "big")
    if test_block(f"{i:02d}".encode(), iv):
        matches.append(i)
    counter += 1
```

Resultado.

```
Q3 candidate ids: ['42']
Q3 block: PLC=42;CMD=SET;N
```

Só uma hipótese dá c_0; a verificação com a chave confirma `PLC=42`. A lógica do ataque não usa a chave: só o oráculo a usa para responder às queries, como o enunciado permite.

IV aleatório. Com IV aleatório e imprevisível, o atacante não sabe o IV da próxima mensagem quando escolhe P, logo não consegue forçar $P \oplus iv = P_0 \oplus iv_0$; o bloco de entrada de E_K é imprevisível. Um IV único mas previsível não chega: a unicidade evita reutilização de IV entre mensagens, mas não impede este ataque de chosen-plaintext, porque o problema é a previsibilidade. Para CBC, o IV deve ser aleatório (ou um nonce imprevisível).

Questão 4

Q4.a. O raciocínio é igual ao da Q1: a parte de confidencialidade do GCM é CTR, por isso $C' = C \oplus P \oplus P'$:

```
Q4 forged: 8fc0cc2e6567e77bcfaad7c8363ae39b2db91d4df9aa5acb74dde650cfe6
Q4 tag:    a903a1a5b290dfe9d41c2634deed9cb1 (inalterada)
```

Mudam outra vez os bytes 24 e 25 (`0x75`→`0x74`, `0xdc`→`0xdd`), como na Q1. Ao decifrar e verificar, o PLC obtém:

```
Q4 forged rejected: InvalidTag
```

O GCM calcula a tag com GHASH sobre o ciphertext (e AAD/comprimentos) e encripta o resultado com $E_K(J_0)$; a tag autentica o ciphertext inteiro. Alterar qualquer byte muda o GHASH e a tag deixa de bater. Assim, embora a parte CTR continue maleável, a verificação rejeita a mensagem antes de ela ser usada. É a autenticação que faltava na Q1, não uma cifra diferente.

Q4.b. Reutilizar (chave, nonce) é fatal nos dois sentidos. Na confidencialidade, o keystream é o mesmo nas duas mensagens, logo $C_1 \oplus C_2 = P_1 \oplus P_2$; conhecendo P_1 (o formato é público) recupera-se P_2. Em código, com uma segunda mensagem cifrada com o mesmo nonce:

```
Q4.b recovered other: PLC=07;CMD=SET;NAOH_PPM=99999;
```

Na autenticação, a tag é $T = E_K(J_0) \oplus \mathrm{GHASH}_H(\cdot)$. Com o mesmo nonce, $E_K(J_0)$ é igual nas duas tags, por isso $T_1 \oplus T_2$ cancela essa parcela e fica só a diferença dos GHASH. Com plaintexts conhecidos, isso dá equações em H (a chave de GHASH); recuperado H, o atacante calcula tags válidas para qualquer mensagem sob aquele nonce, mesmo sem a chave AES. Portanto o nonce tem de ser único por chave (contador ou 96 bits aleatórios); a unicidade é o que separa o GCM de um CTR com MAC.

Conclusão

Os modos de operação não acrescentam integridade por si: CTR continua maleável (Q1) e CBC com IV previsível permite chosen-plaintext (Q3). ECB revela igualdade e estrutura, logo não serve para comandos de formato fixo (Q2). A solução é um modo autenticado (GCM) com nonce único (Q4), e nunca reutilizar (key, nonce).